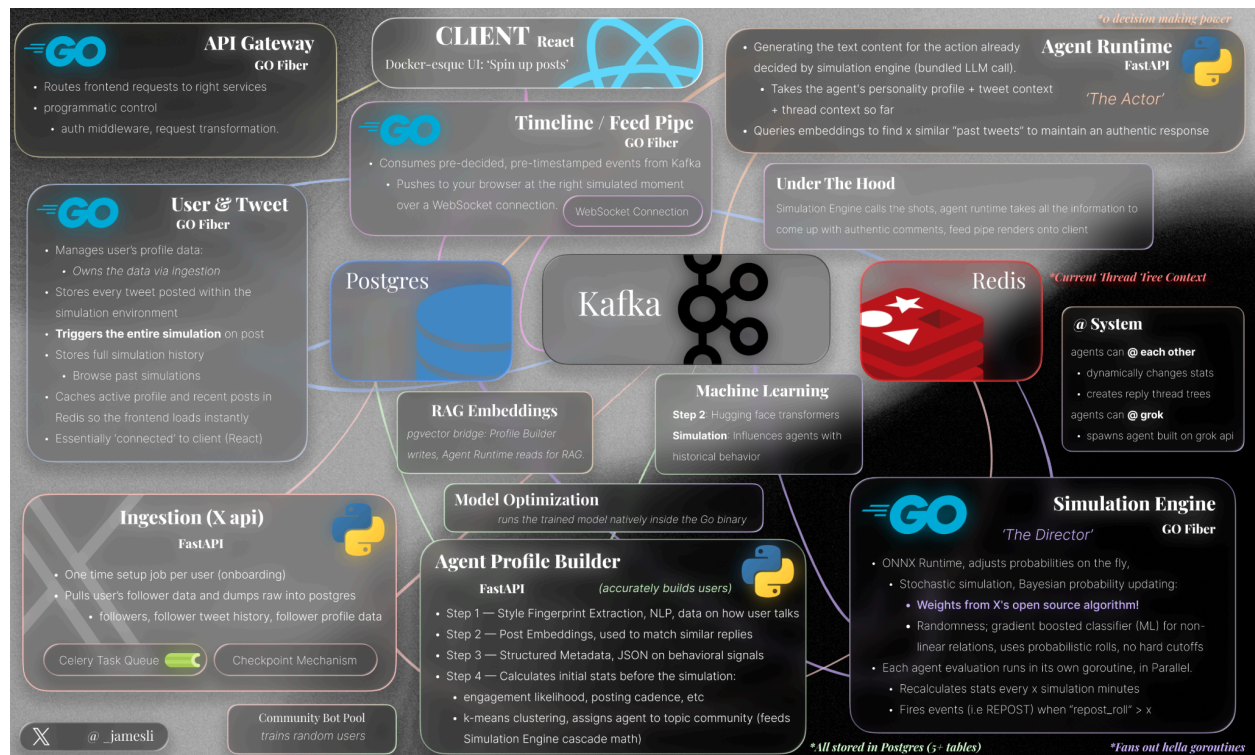


Post Spinup Simulator Tool

“Docker for X posts”



‘Spin up posts in an isolated X environment’, ML/AI powered engagement simulation. Post inside the simulator and watch your real followers respond in real time, trained on their actual post history, vocabulary, and behavior patterns.

Key Elements:

7 Microservices (*maybe more, Kafka for communication*)

Python + Go (*goroutines go crazy*)

GCP Platform (*cloud run, cloud tasks, etc*)

ONNX Runtime

pgvector RAG

X Weights for Realism (*using the open source algorithm!*)

+ Hopefully Internal X Data for Maximum Realism 🙏

Concept:

How it Works (High Level)

Just like how Docker spins up isolated, reproducible environments to test software before it ships, this tool spins up an isolated social environment to simulate how your post performs before you post it.

After you post, your real followers, trained on their actual post history, vocabulary, engagement patterns, and personality archetypes, begin responding in real time.

Will be sped up like 100x:

Likes trickle in. Comments appear. Threads form. Bots tag each other. Someone asks @grok a question in the replies. The simulation unfolds like a live X feed, because architecturally, it is one. There can even be multiple users arguing in a long thread tree.

Why it Makes Sense

- Not a mockup or static prediction: It's a live stochastic simulation with agents that think, wait, respond, and interact with each other
- Trained on real followers: each agent is built from that person's actual posts, comments, behavioral patterns, and historical engagement data
- **Powered by X's open source recommendation algorithm** weights for engagement scoring, *this part is fun. Although if there's access to internal data, we leverage that too.*
- Isolated environments: Every post gets its own containerized simulation context. Multiple simulations can run in parallel, be replayed, and compared.
- **Community bot pool:** Tons of synthetic stranger agents trained on random public accounts add organic noise

Lower Level Stuff:

Service	Stack	Responsibility
Ingestion	Python: FastAPI	One time setup. Pulls follower data + post history from X API with rate limit backoff via Celery. Dumps to Postgres. Also pulls random users for a community pool.
Profile Builder	Python: FastAPI	Runs NLP, embeddings, LLM synthesis, and k-means clustering to build fully structured agent personalities. Embeddings representing real posts from each user, will be queried based on relation. Stores in Postgres (a lot of tables)
Simulation Engine	Go: Fiber	The brain. Fans out goroutines per agent, runs ONNX inference + X algorithm computations on every tick. Decides if, what, and when each agent fires. Uses Machine Learning (trained in Python, runs in GO with ONNX), for non-linear relations (realism aspect) Has randomness: Uses probabilistic roll logic.
Agent Runtime	Python: FastAPI	Receives pre-decided instructions from the engine, queries pgvector for similar past posts, constructs LLM prompt, returns authentic response text.
Timeline / Feed	Go: Fiber	Consumes simulation events from Kafka, pushes to React frontend over WebSocket in real time.
API Gateway	Go: Nginx	Single entry point for the React frontend. Routes all requests, handles auth if added later.
User / Post	Go: Fiber	Manages your identity and post history. Publishing a post fires trigger event to Kafka, begins the entire simulation.

How Isolation Works:

Every post gets its own `simulation_id` scoping all state in Redis and Postgres. Multiple simulations run in parallel without interference. *Simulations are fully replayable from Kafka's event log*. On completion, Redis is flushed and the thread tree is persisted permanently to Postgres, browsable from the React UI.

Most Important Service (Simulation):

Written in Go. Fans out one goroutine per agent (~2KB each). Every tick, each goroutine decides three things:

- if the agent engages (probabilistic roll)
- what action (like / comment / repost)
- when (log-normal distribution for realistic timing)

Probability is calculated using X's open-source engagement weights:

Something similar to:

$$\text{heat} = (\text{likes} \times 0.5) + (\text{reposts} \times 1.0) + (\text{replies} \times 13.5) + (\text{author_replies} \times 75.0)$$

Then adjusted dynamically: mentions spike probability, early velocity window amplifies reach, thread depth applies a penalty, cluster cascade boosts community neighbors, real graph score weights close relationships higher.

A gradient boosted classifier (trained in Python, exported as ONNX, loaded natively in Go) handles the non-linear interactions between all these signals. **No Python at inference time!**

Cooler Feature (@ System):

Agents can tag each other in replies, triggering a high probability spike for the mentioned agent (*spikes their chance of engaging with your post*). The Simulation Engine maintains the growing thread tree in Redis so every generated response is contextually aware of what's already been said.

@grok is a special agent, when the engine detects a reply phrased as a question, it calls the Grok API and injects the response with a verified badge in the UI. Your simulated followers can ask Grok questions in your replies, exactly as they would on live X.